Jeffrey Palermo 9:34 am on August 4, 2008

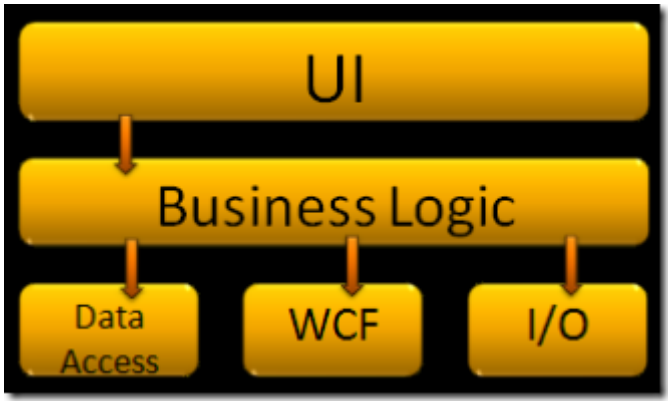
Tags: onion architecture (4)

The Onion Architecture : part 3

Part 1 – Part 2 – This is part 3. part 4. – My RSS feed

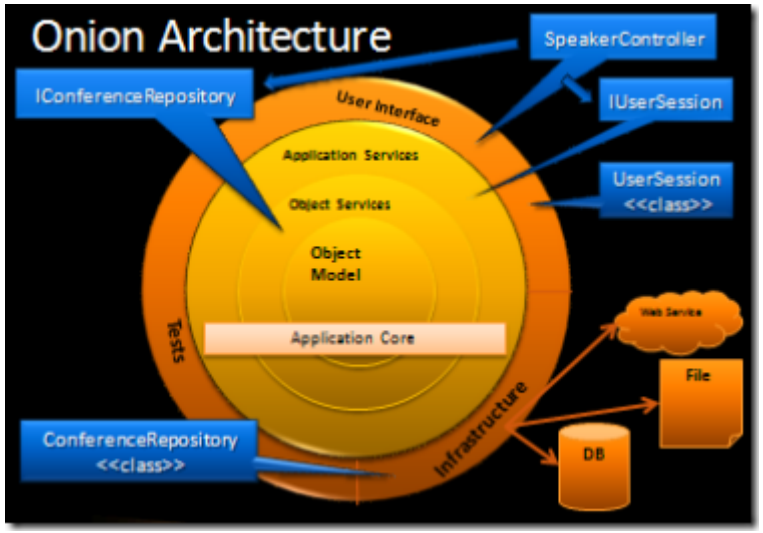
In my previous installments, I described what has become my approach to defining the architecture for an application. Based on feedback, I've modified my diagrams a bit to reduce ambiguity and emphasize key points. The goal of part 3 of this series is to compare and contrast the Onion Architecture with traditional layered architecture. I will flatten the Onion Architecture to see what it looks like compared to traditional layered architecture, and I will force the layered architecture into an onion. Whereas the shape can be either, the structure of the actual application is radically different from what is commonly known and accepted. I'll define four tenets of Onion Architecture at the end.

I must stress again: I am not claiming any breakthroughs in technology or technique. I have learned from other industry thought leaders like Martin Fowler, Ward Cunningham, Kent Beck, Michael Feathers and others (especially those I've had the privilege to work with here in Austin, TX). I'm putting forth the Onion Architecture as an architectural pattern by which we can communicate this radically different architectural approach. Not "radically different as in new". Different as in not mainstream.



Let's review. Traditional layered architecture can look somewhat like the diagram depicted on the right. Each layer communicates with the layer below it. The UI talks to business logic, but it does not talk directly to data access, WCF, etc. The layering approach does call out the need to keep certain categories of code out of the UI. The big downfall is that business logic ends up coupled to infrastructure concerns. Data Access, I/O, and Web Services are all infrastructure. Infrastructure is any code that is a **commodity and does not give your application a competitive advantage**. This code is most likely to change frequently as the application goes through years of maintenance. Web services are still fairly new, and the first version in .Net, ASMX, is already deprecated in favor of WCF. We can be assured that WCF's days are numbered as well, so it is foolish to tightly couple the business logic to WCF. Data access changes every two years or so, so we definitely don't want to

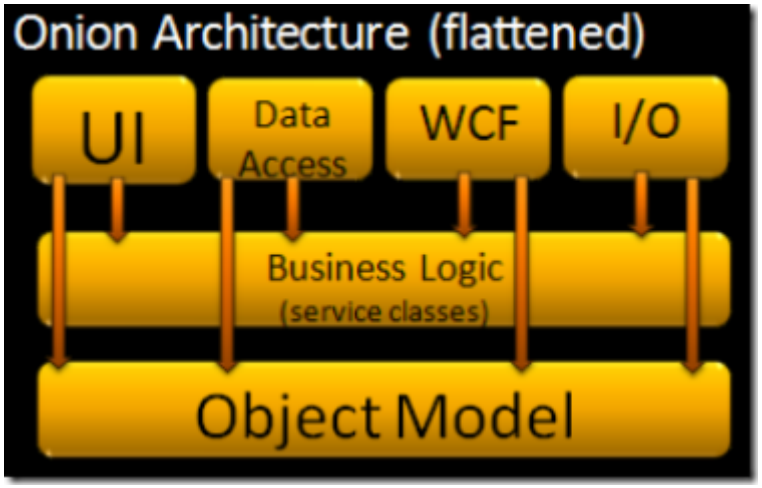
be tightly coupled to it. For long-life, we would want our business logic to be independent of these infrastructure concerns so that as infrastructure changes, the business logic doesn't have to.



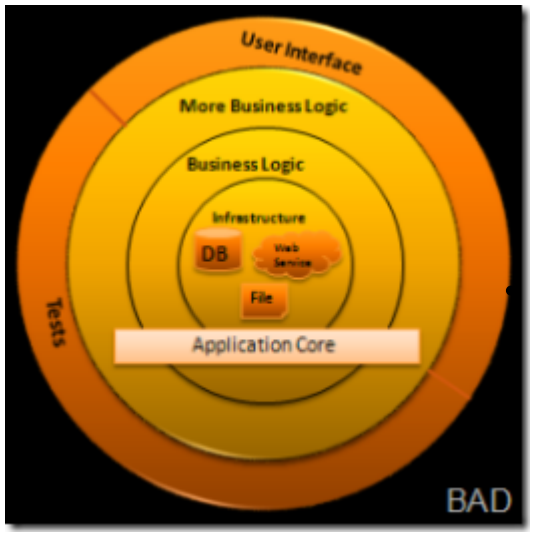
Let's review Onion Architecture. The object model is in the center with supporting business logic around it. The direction of coupling is toward the center. The big difference is that any outer layer can directly call any inner layer. With traditionally layered architecture, a layer can only call the layer directly beneath it. **This is one of the key points that makes Onion Architecture different from traditional layered architecture**. Infrastructure is pushed out to the edges where no business logic code couples to it. The code that interacts with the database will implement interfaces in the application core. The application core is coupled to those interfaces but not the actual data access code. In this way, we can change code in any outer layer without affecting the application core. We include tests because any long-lived application needs tests. Tests sit at the outskirts because the application core doesn't couple to them, but the tests are coupled to the application core. We could also have another layer of tests around the entire outside when we test the UI and infrastructure code.

This approach to application architecture ensures that the application core doesn't have to change as: the UI changes, data access changes, web service and messaging infrastructure changes, I/O techniques change.

To the right, I have created a diagram which attempts to show what Onion Architecture would look like when represented as a traditionally layered architecture. The big difference is that **Data Access is a top layer** along with UI, I/O, etc. Another key difference is that the layers above can use any layer beneath them, not just the layer immediately beneath. Also, business logic is coupled to the object model but not to infrastructure.



To the left here I have attempted to represent traditionally layered architecture using concentric circles. I have used black lines around the layers to denote that each outer layer only talks to the layer immediately toward the center. The big kicker here is that we clearly see the application is built around data access and other infrastructure. Because the application has this coupling, when data access, web services, etc. change, the business logic layer will have to change. The world view difference is how to handle infrastructure. Traditional layered architecture couples directly to it. Onion Architecture pushes it off to the side and defines abstractions (interfaces) to depend on.



Then the infrastructure code also depends on these abstractions (interfaces). Depending on abstractions is an old principle, but the Onion Architecture puts that concepts right up front.

Key tenets of Onion Architecture:

The application is built around an independent object model
Inner layers define interfaces. Outer layers implement interfaces
Direction of coupling is toward the center
All application core code can be compiled and run separate from infrastructure

I encourage you to use the term "Onion Architecture" when speaking about architectures that adhere to the above four tenets. I believe that this approach to architecture leads to long-lived systems that are easy to maintain. Also, in my experience, this architecture yields dividends soon after a project starts since it makes the code a breeze to change.

Although I don't call out an IoC container as a key tenet, when using a mainstream language like Java or C#, an IoC container makes the code fit together very easily. Some languages have IoC features built-in, so this is not always necessary. If you are using C#, I highly recommend using Castle Windsor or StructureMap.

> Not sure if this is architecture is any different from the traditional layered architecture. The driving force behind the Onion architecture, as I understand, is for the business logic/object domain to not depend on the infrastructure for ex.dataaccess.
> We need to clarify what "depend" means. The dependency that we are talking about in Onion architecture is "where does the interface definitions reside that outer layers need to implement" whereas the dependency in the traditional layered architecture is "what does it take to make the application run".
> So if I define and host all the interfaces for my system in the business layer then what is the difference between the 2 architectures since even in Onion architecture you still need your infra services (dataaccess) to be up and running for your system to work?
> Obviously in any architecture the business logic does depend on the dataaccess layer to get and put data (no brainer). Question is how is dependecy achieved? Ok, one simple way is to define an interface for dataaccess and code against the interface in the

business layer, a logical place to house the interface would be in the business layer assembly or a separate assembly. Now the dataaccess implements the interface. Tommorrow the database technology changes I would simply create another implementation of the interface and allow the business layer to use it using IOC (Another big hue and cry about the coolness of IOC..why make it a design pattern instead simply call it a cool tip…nevermind thats just me…. where instead of hardcoding the reference to the dataccess assembly in the business layer you move it to a config file..how many times havent you saved stuff that could potenially change in the future in a config file?).

With reference to your example where you have the business layer depending on dataaccess, wcf and I/O…I would make the business layer depend (via interface) on the dataaccess layer only and delegate to the dataaccess layer how it communicates to the rest of the world to get the "data" say via database, wcf, i/o, bapi etc, so if wcf changes tommorrow business layer doesnt need to change.

So far I have stuck to the traditional layer architecture (in mind atleast) without having to use anything from the Onion architecture.

The other tenet that is mentioned above is "All application core code can be compiled and run separate from infrastructure".

So if I ask the question, in the traditional layered approach can the application run without the dataacess layer answer is nope.

Again, in the onion architecture approach can the application run without the dataacess layer answer is nope. How are you going to run an application without the help of the infra (database).

According to ’Hexagonal Architecture” make use of a mock database…in a complex system where I depend on data from mutilple datasources (ERP, home grown logisitic system, file – csv and text files, multiple sql databases) I am not clear how a mock database is applicable. I cant potenially create a mock database which aggregates all my datasources so that I can run my tests against the business layer without having my dataaccess layer "up and running".

As long as we make sure that the business logic does not seep into other layers is there a difference between the 2 architectures?

Comments are closed.

← A must-read blog

Follow me on twitter @jeffreypalermo →